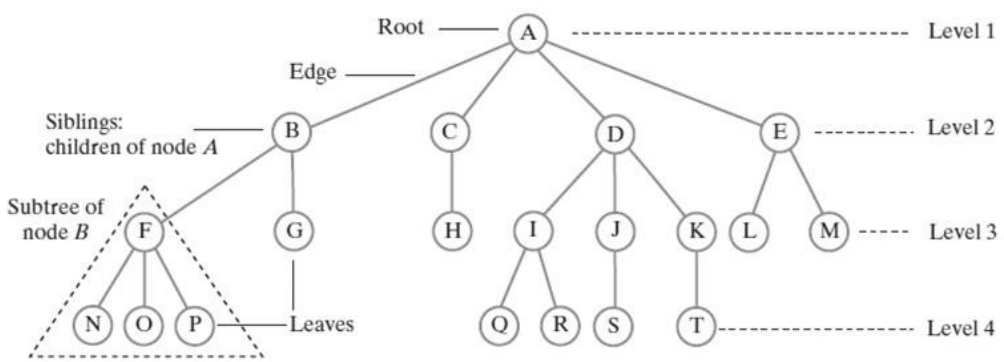
PKG 18: TREES and GOOGLE TREES

Data Structures and Abstractions with Java, Frank M. Carrano and Timothy M. Henry:

Trees

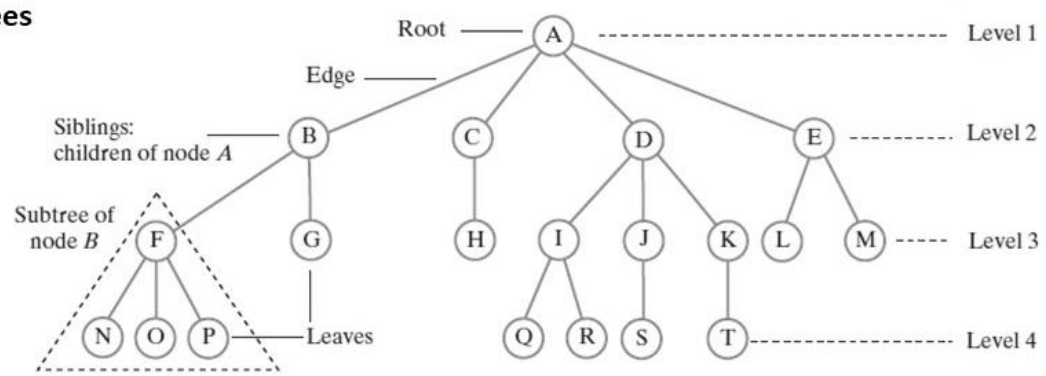


- **Path**: Begins at the root and goes from node to node along the edges that join them.
  → Path between the root and any node is **unique**.

- **Length of a Path**: The number of edges that compose it → A to N has length 3.

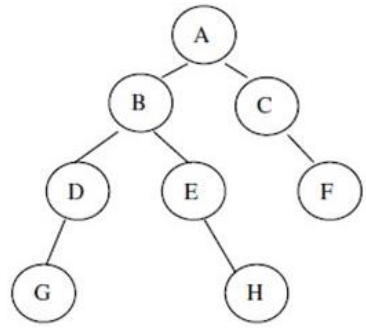
- **Height/Depth of a tree**: the number of level in the tree → A empty tree has height 0.

Trees

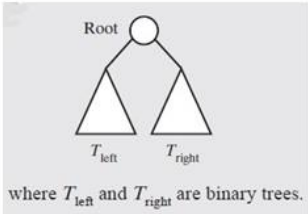


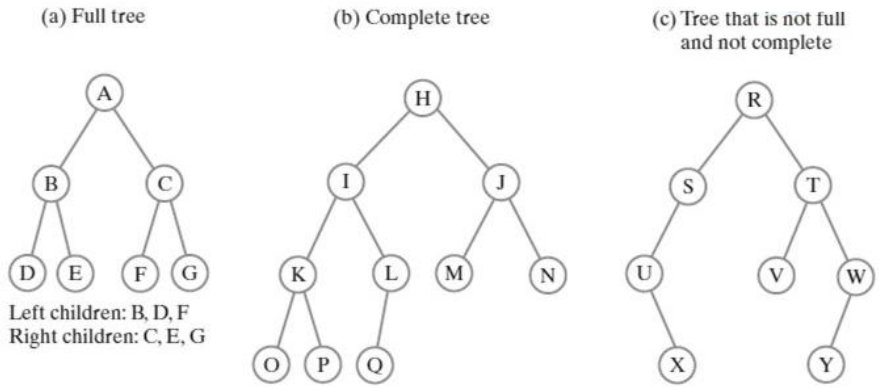
- **General Tree**: Each node can have an **arbitrary number** of children.

- **Binary Tree**: Each node has **at most two** children.



Binary Tree



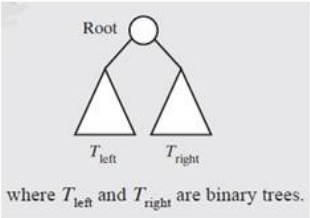


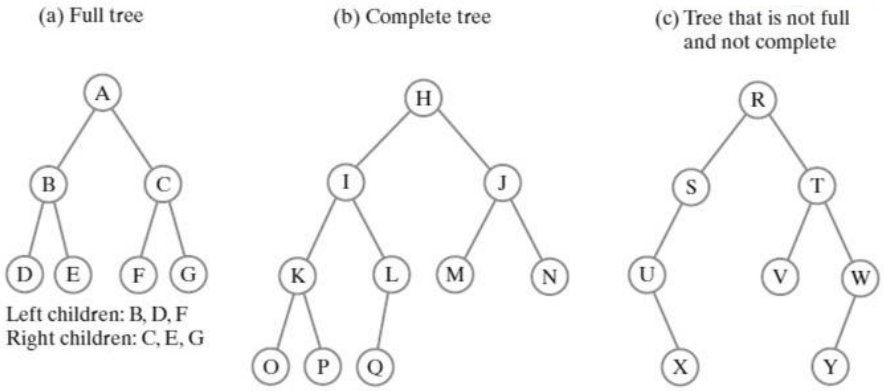
- **Binary Tree**: Each node has at most two children: **Left child** and **Right child**

- Every subtree in a binary tree is also a binary tree
- (a): Left children B, D, E and Right children C, F, G

- (a): A **full** tree:
  - Height: h
  - All leaves are on a same level, level h
  - Every nonleaf has exactly 2 children

Binary Tree



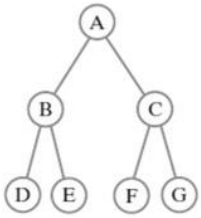


- (a) **Completely balanced**: each node has two subtrees whose heights are exactly the same. Competely balanced trees are full.

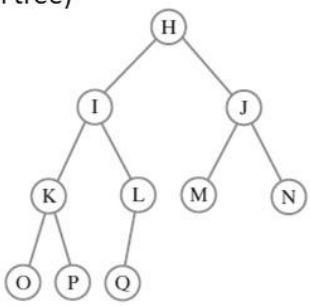
- (b): **Height balanced/balanced**: the subtrees of each node differ in height by no more than 1.

- **Balanced Node**: A node in a binary tree whose subtrees differ in height by no more than 1 is known as balanced node. All nodes in a balanced binary tree are balanced.

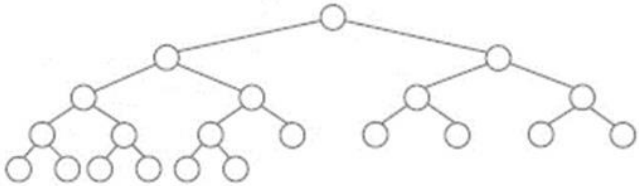
- A **full** tree:
  - Height: h
  - All leaves are on a same level, level h
  - Every non-leaf has exactly 2 children



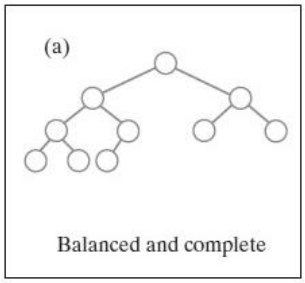
- A **complete** tree:
  - Tree is **full** to its **next-to-last** level
  - Leaves on the last level are filled from left to right
  - The last level does not have to be full (vs. a full tree)

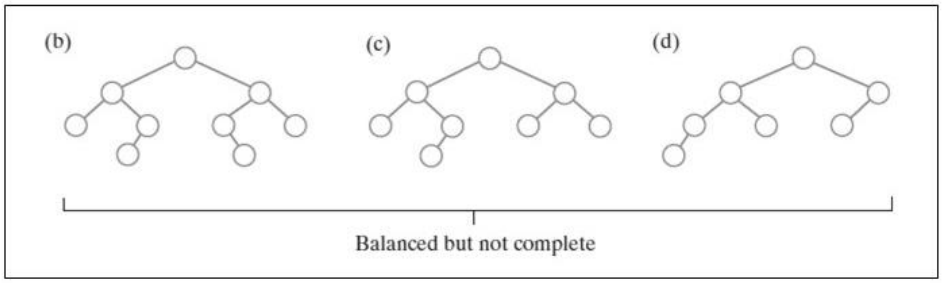


- A **balanced/height balanced** tree: the subtrees of each node differ in height by no more than 1.

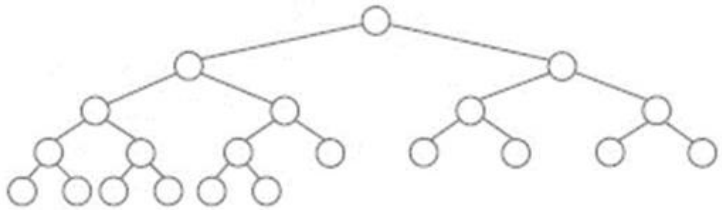


Balanced and complete **vs.** Balanced and not complete = check left/right



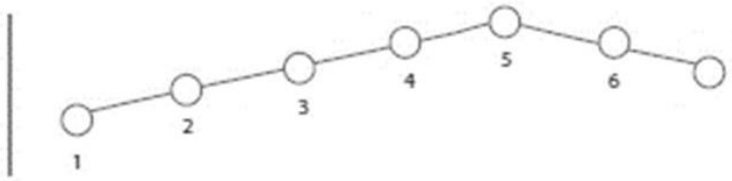


B1. --- What is the **height** of the shortest **binary tree** that contains 21 nodes? Is this tree **full**? Is it **balanced**?



height = 5, balanced and complete

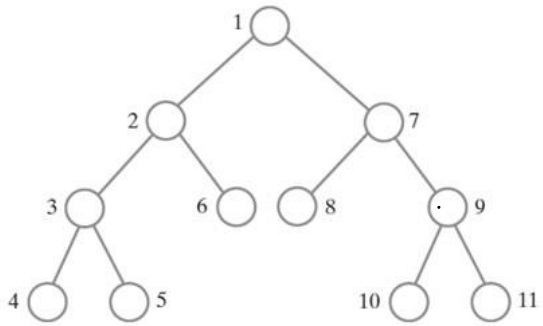
B2. --- Suppose that you draw **a binary tree** so that **no two nodes align vertically**. Demonstrate that a vertical line moving from left to right across the tree crosses the nodes in the same order in which an **inorder traversal** visits nodes.



Text

B3. --- Consider a traversal of a binary tree. Suppose that visiting a node means to simply display the data in the node. What are the results of each of the following traversals of the tree in Figure A.1?

Binary Tree Traversals, Preorder Traversal



- **Pre** = Visit the root **Before** we visit the root's subtrees

recursion

1. Visit the **root**
2. Visit all the nodes in the root's left subtree
3. Visit all the nodes in the root's right subtree

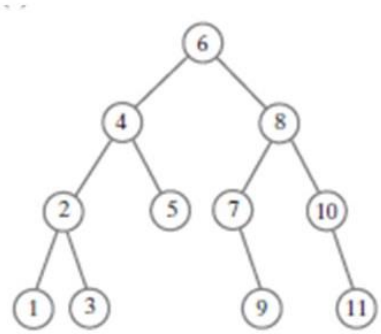
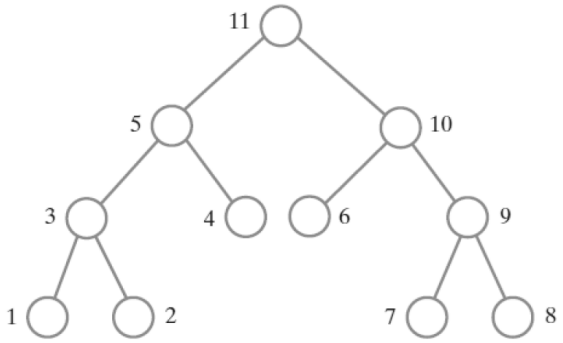


Figure **A.1**

Preorder: **R L R**

Preorder: **6** **4** **2**-1-3 **5**-*-* **8** **7**-*-9 **10**-*-11

Binary Tree Traversals, Postorder Traversal



- **Post** = Visit the root **After** we visit the root's subtrees

1. Visit all the nodes in the root's left subtree
2. Visit all the nodes in the root's right subtree
3. Visit the **root**

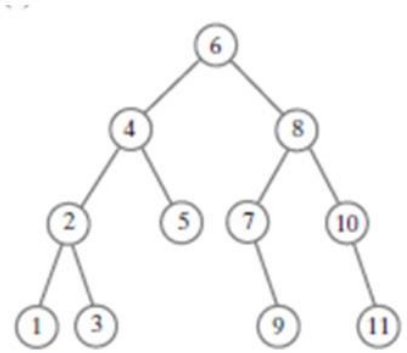
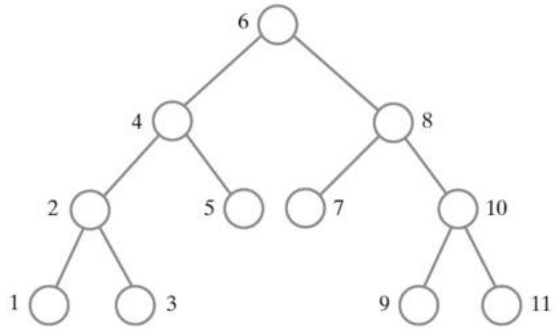


Figure **A.1**

Postorder: **L R R**

Postorder: 1-3-**2** *-*-**5** **4** *-9-**7** *-11-**10** **8** **6**

Binary Tree Traversals, Inorder Traversal



- **In** = Visit the root **Between** we visit the root's subtrees

1. Visit all the nodes in the root's left subtree
2. Visit the **root**
3. Visit all the nodes in the root's right subtree

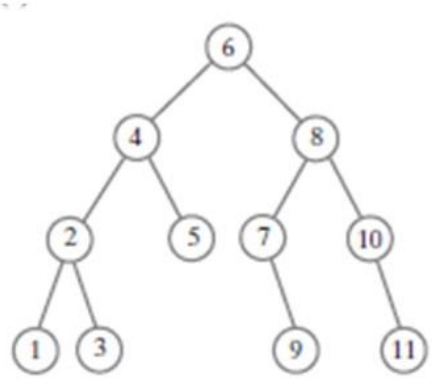
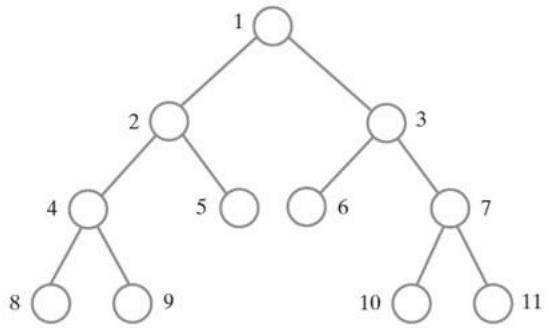


Figure **A.1**

Inorder: **L R R**

Inorder: 1-**2**-3 **4** *-**5**-* **6** *-**7**-9 **8** *-10-11

Binary Tree Traversals, Level-Order Traversal



- **Level** = Visit an entire level before moving to the next level

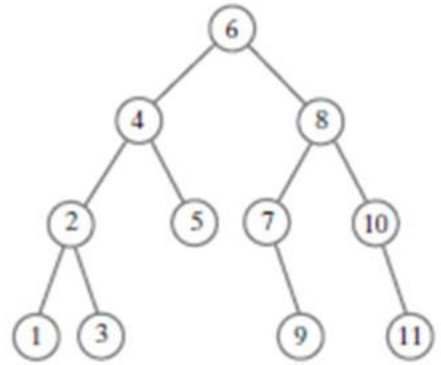


Figure **A.1**

Inorder: **Level to Level**

Inorder: 6 4-8 2-5-7-10 1-3-9-11

B4. --- Consider a traversal of a binary tree. Suppose that visiting a node means to simply display the data in the node. What are the results of each of the following traversals of the tree in Figure A.2?

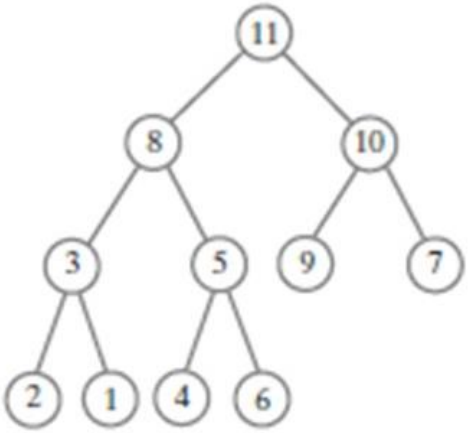


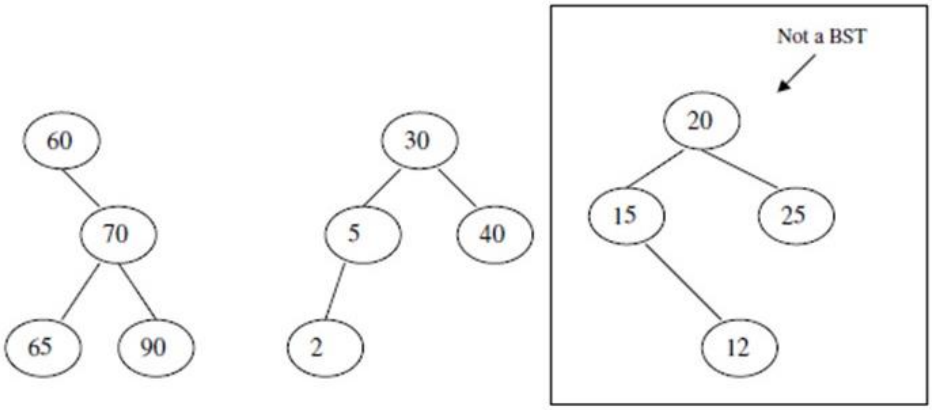
Figure **A.2**

B5. --- The two trees in Figure A.1 and Figure A.2 contain integer data.

a. Is the tree in Figure A.1 **a binary search tree**? Why or why not?

b. Is the tree in Figure A.2 **a maxheap**? Why or why not?

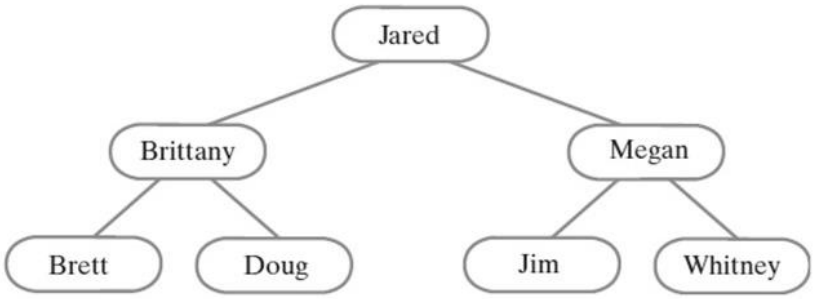
Binary Search Tree (BST)



- **A Binary Search Tree is a binary tree that satisfies the following 3 properties**

  1. Each node has a **Comparable** object with unique key (or search key)
  2. All keys in **left** subtree **<** key in **the root <** All keys in **right** subtree
  3. **Both** left subtree and right subtree are **binary search trees**

Binary Search Tree (BST)

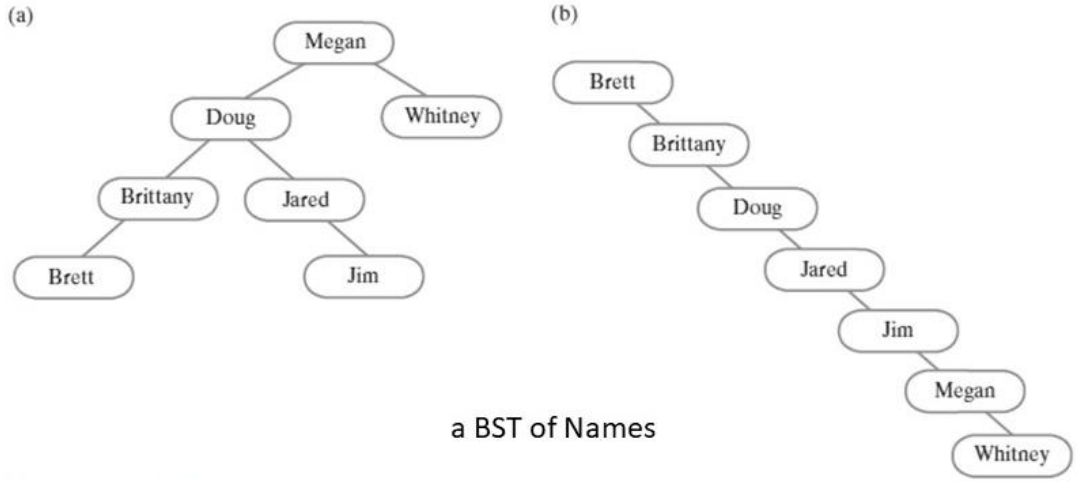


a BST of Names

- **A Binary Search Tree is a binary tree that satisfies the following 3 properties**

  1. Each node has a **Comparable** object with unique key (or search key)
  2. All keys in **left** subtree **<** key in **the root <** All keys in **right** subtree
  3. **Both** left subtree and right subtree are **binary search trees**

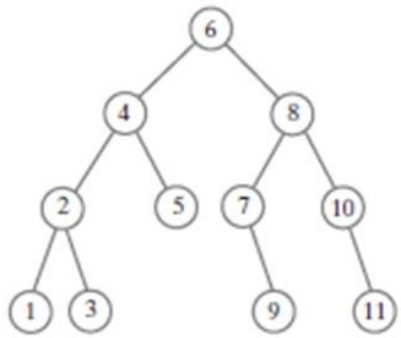
Binary Search Tree (BST)



a BST of Names

- **A Binary Search Tree is a binary tree that satisfies the following 3 properties**

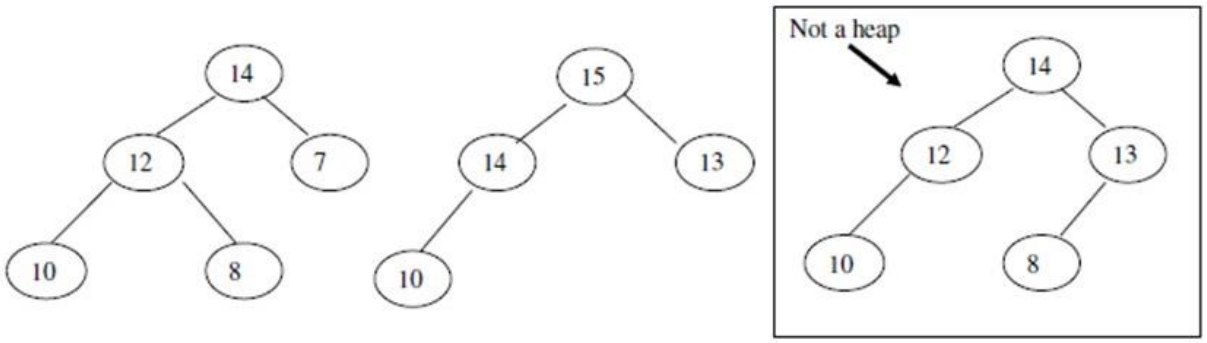
  1. Each node has a **Comparable** object with unique key (or search key)
  2. All keys in **left** subtree **<** key in **the root <** All keys in **right** subtree
  3. **Both** left subtree and right subtree are **binary search trees**



Binary Search Tree?

Figure **A.1**

Heap



- **Heap:**
  - A **complete** binary tree
  - Nodes contain **Comparable** objects
  - Each node contains no smaller (or no larger) than objects in its descendants

- **Maxheap :** Object in a node is ≥ its descendant objects
- **Minheap :** Object in a node is ≤ descendant objects

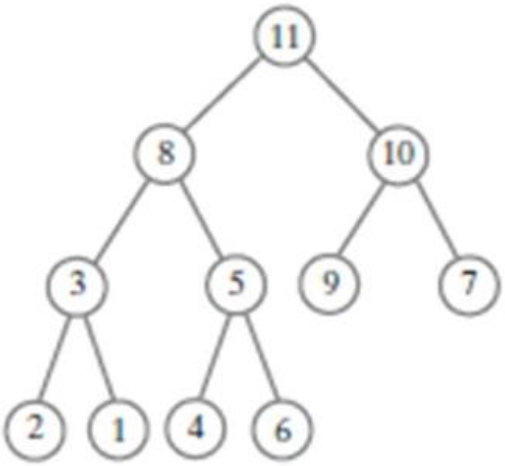


Figure **A.2**

Max Heap?

TREE IMPLEMENTATION

```
class BinaryNode<T> {

    private T data;
    private BinaryNode<T> leftChild;  // Reference to left child
    private BinaryNode<T> rightChild; // Reference to right child

    public BinaryNode() {
        this(null); // Call next constructor
    } // end default constructor

    public BinaryNode(T dataPortion) {
        this(dataPortion, null, null); // Call next constructor
    } // end constructor

    public BinaryNode(T dataPortion, BinaryNode<T> newLeftChild,
            BinaryNode<T> newRightChild) {
        data = dataPortion;
        leftChild = newLeftChild;
        rightChild = newRightChild;
    } // end constructor

    public T getData() {
        return data;
    } // end getData

    public void setData(T newData) {
        data = newData;
    } // end setData

    public BinaryNode<T> getLeftChild() {
        return leftChild;
    } // end getLeftChild

    public void setLeftChild(BinaryNode<T> newLeftChild) {
        leftChild = newLeftChild;
    } // end setLeftChild

    public boolean hasLeftChild() {
        return leftChild != null;
    } // end hasLeftChild

    public BinaryNode<T> getRightChild() {
        return rightChild;
    } // end getRightChild

    public void setRightChild(BinaryNode<T> newRightChild) {
        rightChild = newRightChild;
    } // end setRightChild

    public boolean hasRightChild() {
        return rightChild != null;
    } // end hasRightChild

    public boolean isLeaf() {
        return (leftChild == null) && (rightChild == null);
    } // end isLeaf
```

TREE IMPLEMENTATION

```
    public int getNumberOfNodes() {
        int leftNumber = 0;
        int rightNumber = 0;

        if (leftChild != null) {
            leftNumber = leftChild.getNumberOfNodes();
        }

        if (rightChild != null) {
            rightNumber = rightChild.getNumberOfNodes();
        }

        return 1 + leftNumber + rightNumber;
    } // end getNumberOfNodes

    public int getHeight() {
        return getHeight(this); // Call private getHeight
    } // end getHeight

    private int getHeight(BinaryNode<T> node) {
        int height = 0;

        if (node != null) {
            height = 1 + Math.max(getHeight(node.getLeftChild()),
                    getHeight(node.getRightChild()));
        }

        return height;
    } // end getHeight

    public BinaryNode<T> copy() {
        BinaryNode<T> newRoot = new BinaryNode<>(data);

        if (leftChild != null) {
            newRoot.setLeftChild(leftChild.copy());
        }

        if (rightChild != null) {
            newRoot.setRightChild(rightChild.copy());
        }
        return newRoot;
    } // end copy
} // end BinaryNode
```

TREE IMPLEMENTATION

```
public interface GeneralTreeInterface<T> extends TreeInterface<T>,
        TreeIteratorInterface<T> {
    public void setTree(T rootData);
    public void setTree(T rootData, GeneralTreeInterface<T>[] children);
} // end GeneralTreeInterface
```

TREE IMPLEMENTATION

```
public interface TreeInterface<T> {
    public T getRootData();
    public int getHeight();
    public int getNumberOfNodes();
    public boolean isEmpty();
    public void clear();
} // end TreeInterface
```

TREE IMPLEMENTATION

```
public interface TreeIteratorInterface<T> {
    public Iterator<T> getPreorderIterator();
    public Iterator<T> getPostorderIterator();
    public Iterator<T> getInorderIterator();
    public Iterator<T> getLevelOrderIterator();
} // end TreeIteratorInterface
```

TREE IMPLEMENTATION

```
import java.util.Iterator;
import java.util.NoSuchElementException;

public class GeneralTree<T> implements GeneralTreeInterface<T> {

    private BinaryNode<T> root;

    public GeneralTree() {
        root = null;
    } // end default constructor

    public GeneralTree(T rootData) {
        root = new BinaryNode<>(rootData);
    } // end constructor

    public GeneralTree(T rootData, GeneralTree<T>[] children) {
        privateSetTree(rootData, children);
    } // end constructor

    @Override
    public void setTree(T rootData) {
        root = new BinaryNode<>(rootData);
    } // end setTree

    @Override
    public void setTree(T rootData, GeneralTreeInterface<T>[] children) {
        privateSetTree(rootData, children);
    } // end setTree

    private void privateSetTree(T rootData, GeneralTreeInterface<T>[] children) {
        root = new BinaryNode<>(rootData);

        // Find the first valid child and put it on the left of the root
        int i = 0;
        boolean found = false;
        BinaryNode<T> childList = null;

        while (i < children.length && !found) {
            // Force the child to be one of our trees
            GeneralTree<T> child = (GeneralTree<T>) children[i];

            if ((child != null) && !child.isEmpty()) {
                // Found it
                found = true;
                root.setLeftChild(child.root.copy());
                childList = (BinaryNode<T>) root.getLeftChild();
            } // end if
            i++;
        }
```

```
        while (i < children.length) {
            GeneralTree<T> child = (GeneralTree<T>) children[i];
            if ((child != null) && !child.isEmpty()) {
                childList.setRightChild(child.root.copy());
                childList = (BinaryNode<T>) childList.getRightChild();
            } // end if
            i++;
        } // end while
    } // end privateSetTree

    @Override
    public T getRootData() {
        T rootData = null;
        if (root != null) {
            rootData = root.getData();
        }

        return rootData;
    } // end getRootData

    @Override
    public boolean isEmpty() {
        return root == null;
    } // end isEmpty

    @Override
    public void clear() {
        root = null;
    } // end clear

    protected void setRootData(T rootData) {
        root.setData(rootData);
    } // end setRootData

    protected void setRootNode(BinaryNode<T> rootNode) {
        root = rootNode;
    } // end setRootNode

    protected BinaryNode<T> getRootNode() {
        return root;
    } // end getRootNode

    @Override
    public int getHeight() {
        return root.getHeight();
    } // end getHeight

    @Override
    public int getNumberOfNodes() {
        return root.getNumberOfNodes();
    } // end getNumberOfNodes

    @Override
    public Iterator<T> getPreorderIterator() {
        return new PreorderIterator();
    } // end getPreorderIterator

    @Override
    public Iterator<T> getPostorderIterator() {
        return new PostorderIterator();
    } // end getPostorderIterator
```

```
    @Override
    public Iterator<T> getInorderIterator() {
        throw new UnsupportedOperationException();
    } // end getInorderIterator

    @Override
    public Iterator<T> getLevelOrderIterator() {
        throw new UnsupportedOperationException();
    } // end getLevelOrderIterator

    private class PreorderIterator implements Iterator<T> {

        private StackInterface<BinaryNode<T>> nodeStack;

        public PreorderIterator() {
            nodeStack = new LinkedStack<>();
            if (root != null) {
                nodeStack.push(root);
            }
        } // end default constructor

        @Override
        public boolean hasNext() {
            return !nodeStack.isEmpty();
        } // end hasNext

        @Override
        public T next() {
            T result = null;

            if (nodeStack.isEmpty()) {
                throw new NoSuchElementException();
            } else {
                BinaryNode<T> top = nodeStack.pop();
                result = top.getData();

                // Push the children on the stack, right then left.
                if (top.hasRightChild()) {
                    nodeStack.push(top.getRightChild());
                }
                if (top.hasLeftChild()) {
                    nodeStack.push(top.getLeftChild());
                }
            }

            return result;
        } // end next

        @Override
        public void remove() {
            throw new UnsupportedOperationException();
        } // end remove
    } // end PreorderIterator

    private class PostorderIterator implements Iterator<T> {

        private StackInterface<PostOrderNode> nodeStack;
```

```
        public PostorderIterator() {
            nodeStack = new LinkedStack<>();
            if (root != null) {
                nodeStack.push(new PostOrderNode(root, PostOrderState.CHILDREN));
            }
        } // end default constructor

        @Override
        public boolean hasNext() {
            return !nodeStack.isEmpty();
        } // end hasNext

        @Override
        public T next() {
            T result = null;
            if (nodeStack.isEmpty()) {
                throw new NoSuchElementException();
            } else {
                PostOrderNode top = nodeStack.pop();
                PostOrderState state = top.state;

                while (state != PostOrderState.TOP) {
                    assert state == PostOrderState.CHILDREN;
                    StackInterface<PostOrderNode> children = new LinkedStack<>();
                    BinaryNode<T> child = (BinaryNode<T>) top.node.getLeftChild();
                    while (child != null) {
                      children.push(new PostOrderNode(child, PostOrderState.CHILDREN));
                        child = (BinaryNode<T>) child.getRightChild();
                    }

                    // push the parent and the children on the stack
                    nodeStack.push(new PostOrderNode(top.node, PostOrderState.TOP));
                    while (!children.isEmpty()) {
                        nodeStack.push(children.pop());
                    }

                    top = nodeStack.pop();
                    state = top.state;
                }
                result = top.node.getData();
            }

            return result;
        } // end next

        @Override
        public void remove() {
            throw new UnsupportedOperationException();
        } // end remove
    } // end PostorderIterator

    private enum PostOrderState {
        TOP, CHILDREN
    };

    private class PostOrderNode {

        public BinaryNode<T> node;
        public PostOrderState state;

        PostOrderNode(BinaryNode<T> theNode, PostOrderState theState) {
```

```
            node = theNode;
            state = theState;
        }
    }
} // end GeneralTree
```

TREE IMPLEMENTATION

```
import java.util.Iterator;

public class GeneralTreeDiver {
    public static void main(String[] args) {
        // Tree
        System.out.println("         Carole");
        System.out.println("        /   |  \\");
        System.out.println("       /    |   \\");
        System.out.println("      /     |    \\");
        System.out.println("   Brett Jennifer Brian");
        System.out.println("    /     / \\");
        System.out.println("   /     /   \\");
        System.out.println("  /     /     \\");
        System.out.println("Susan Jared  Jamie ");
        System.out.println();

        // Leaves
        GeneralTreeInterface<String> brianTree = new GeneralTree<>("Brian");
        GeneralTreeInterface<String> susanTree = new GeneralTree<>("Susan");
        GeneralTreeInterface<String> jaredTree = new GeneralTree<>("Jared");
        GeneralTreeInterface<String> jamieTree = new GeneralTree<>("Jamie");

        // Subtrees:
        // The casts are safe because the new arrays contains null entries
        @SuppressWarnings("unchecked")
        GeneralTreeInterface<String>[] brettChildren = new GeneralTree[2];
        brettChildren[0] = susanTree;
        brettChildren[1] = null;
        GeneralTreeInterface<String> brettTree = new GeneralTree<>();
        brettTree.setTree("Brett", brettChildren);

        @SuppressWarnings("unchecked")
        GeneralTreeInterface<String>[] jenniferChildren = new GeneralTree[2];
        jenniferChildren[0] = jaredTree;
        jenniferChildren[1] = jamieTree;
        GeneralTreeInterface<String> jenniferTree = new GeneralTree<>();
        jenniferTree.setTree("Jennifer", jenniferChildren);

        @SuppressWarnings("unchecked")
        GeneralTreeInterface<String>[] caroleChildren = new GeneralTree[3];
        caroleChildren[0] = brettTree;
        caroleChildren[1] = jenniferTree;
        caroleChildren[2] = brianTree;
        GeneralTreeInterface<String> caroleTree = new GeneralTree<>();
        caroleTree.setTree("Carole", caroleChildren);

        showTreeStats(caroleTree, "carole", 7);
        testTraversals(caroleTree, "carole brett susan jennifer jared jamie brian",
                "susan brett jared jamie jennifer brian carole");

        testEmptyTree();

        System.out.println("Done.");
    }  // end main

```

```
    public static void testTraversals(GeneralTreeInterface<String> aTree,
            String preorder, String postorder) {
        testPreorder(aTree, preorder);
        testPostorder(aTree, postorder);
    } // end testTraversals

    public static void showTreeStats(GeneralTreeInterface<String> tree,
            String root, int numberOfNodes) {
        if (tree.isEmpty()) {
            System.out.println("The tree is empty");
        } else {
            System.out.println("The tree is not empty");
        }

        System.out.println("Root of tree is " + tree.getRootData() + "; s/b " + root);

        System.out.println("# nodes in tree is " + tree.getNumberOfNodes()
                + "; s/b " + numberOfNodes);
    } // end showTreeStats

    public static void testPreorder(GeneralTreeInterface<String> tree, String answer)
{
        System.out.println("\nPreorder:");
        System.out.println(answer + "  Expected");

        Iterator<String> preorder = tree.getPreorderIterator();

        while (preorder.hasNext()) {
            System.out.print(preorder.next() + " ");
        } // end while

        System.out.println(" Actual using PreorderIterator");
        System.out.println("---------------");
    } // end testPreorder

    public static void testPostorder(GeneralTreeInterface<String> tree, String answer)
{
        System.out.println("\nPostOrder:");
        System.out.println(answer + "  Expected");

        Iterator<String> postorder = tree.getPostorderIterator();

        while (postorder.hasNext()) {
            System.out.print(postorder.next() + " ");
        } // end while

        System.out.println(" Actual using PostorderIterator\n---------------");
    } // end testPostorder

    public static void testEmptyTree() {
        System.out.println("\nTest empty tree:");
        GeneralTreeInterface<String> myTree = new GeneralTree<>();
        // myTree.root is null, so myTree is empty, not null

        if (myTree == null) {
            System.out.println("myTree is null--ERROR");
        } else {
            System.out.println("myTree is not null--CORRECT.");

            if (myTree.isEmpty()) {
                System.out.println("myTree is empty--CORRECT.");
```

```
            } else {
                System.out.println("myTree.root is not empty--ERROR");
            }
        } // end if
        System.out.println();
    } // end testEmptyTree
}  // end Driver General Tree



/*
         Carole
        /   |  \
       /    |   \
      /     |    \
   Brett Jennifer Brian
    /     / \s
   /     /   \
  /     /     \
Susan Jared  Jamie

The tree is not empty
Root of tree is Carole; s/b carole
# nodes in tree is 7; s/b 7

Preorder:
carole brett susan jennifer jared jamie brian  Expected
Carole Brett Susan Jennifer Jared Jamie Brian  Actual using PreorderIterator
---------------

PostOrder:
susan brett jared jamie jennifer brian carole  Expected
Susan Brett Jared Jamie Jennifer Brian Carole  Actual using PostorderIterator
---------------

Test empty tree:
myTree is not null--CORRECT.
myTree is empty--CORRECT.

Done.

 */
```